Below is a **full, production-quality** `README.md` for **Terraform Variables** — written for **intermediate to advanced learners**, including **deep explanations, real-world analogies, use cases, limitations, interview questions, and hands-on challenges using only built-in Terraform providers**.

Terraform Variables — In-Depth Guide (README.md)

Terraform variables allow you to **parameterize** your infrastructure code. They make your configuration **dynamic**, **reusable**, and **environment-agnostic**.

This guide covers:

- What variables are and why they matter
- Types of variables
- Advanced features like validations, sensitive variables, dynamic defaults
- Use cases and limitations
- Real-world analogies
- Hands-on challenges
- Interview questions

---

1. What Are Terraform Variables?

Variables in Terraform are **inputs passed into a configuration**. Instead of hardcoding values (like AMI IDs, region names, instance sizes), variables allow you to **externalize configuration** so you can:

- ✔ Reuse the same Terraform code across environments (dev/stage/prod)
- ✔ Decouple infrastructure logic from environment-specific values
- ✔ Securely inject secrets
- ✔ Override values through CLI, tfvars files, or environment variables

---

2. Variable Syntax

A typical variable block:

```
variable "instance_type" {
  type        = string
  default     = "t3.micro"
  description = "Instance type for EC2"
}
```

Using the variable:

```
instance_type = var.instance_type
```

🧪 3. Variable Types (Deep Explanation)

Terraform supports both **primitive** and **complex** types.

3.1 Primitive Types

Type	Example
`string`	"t3.micro"
`number`	2
`bool`	true

Example:

```
variable "enable_logs" {
  type = bool
}
```

3.2 Complex Types

List Type

Ordered collection of values.

```
variable "cidrs" {
  type = list(string)
}
```

Map Type

Key-value pairs.

```
variable "tags" {
  type = map(string)
}
```

Object Type

Structured data.

```
variable "server" {
  type = object({
    name = string
    port = number
  })
}
```

Tuple Type

Different data types allowed.

```
variable "mix" {
  type = tuple([string, number, bool])
}
```

🧭 4. Variable Precedence (Important in Interviews)

Terraform loads variables in the following order (lowest → highest priority):

1️⃣ Default values in variable blocks
2️⃣ `.tfvars` file
3️⃣ Environment variables (`TF_VAR_name=value`)
4️⃣ `-var` flag in CLI
5️⃣ `-var-file` flag

🔑 **Rule:** *Latest wins — highest precedence overrides all.*

🔐 5. Sensitive Variables

Prevents displaying values in output.

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

✔ 6. Variable Validation (Advanced Topic)

Terraform allows custom validation:

```
variable "instance_count" {
  type = number

  validation {
    condition     = var.instance_count > 0 && var.instance_count < 10
    error_message = "Instance count must be between 1 and 9."
  }
}
```

🌀 7. Dynamic Default Values Using Locals

```
locals {
  environment = terraform.workspace
}

variable "instance_type" {
  type = string
  default = local.environment == "prod" ? "t3.large" : "t3.micro"
}
```

🧳 8. Use Cases of Terraform Variables

📌 A. Dev/Stage/Prod Environments

Reuse same code with different:

- Instance sizes
- Regions
- VPC IDs
- RDS sizes

📌 B. CI/CD Integration

You can inject variables at runtime using GitHub Actions or Jenkins.

📌 C. Security and Secrets

Sensitive variables help store:

- DB passwords
- API keys
- Encryption keys

📌 D. Multi-Cloud Deployments

Keep cloud provider selection dynamic:

```
variable "cloud" {
  type = string
}

locals {
  provider = var.cloud == "aws" ? aws : azurerm
}
```

🚫 9. Limitations of Terraform Variables

Limitation	Explanation
No runtime manipulation	Variables are static once Terraform starts.
No loops inside variable blocks	Loops allowed only in locals or resources.
Cannot fetch remote secrets without integration	Requires external provider like AWS Secrets Manager / Vault.
Complex variable types become difficult to read	Objects/tuples can make code harder for beginners.

🏪 10. Real-World Analogy

🎬 Think of Terraform Variables Like Movie Production Parameters

A movie script stays the same, but:

Environment	Actor	Location	Budget
Bollywood	Hritik Roshan	Mumbai	50 Cr

Environment	Actor	Location	Budget
Hollywood	Chris Evans	New York	200M

The script = Terraform configuration
The actors/locations/budget = variables

You run the **same script** → but change values to produce different outcomes.

🛠️ 11. Hands-On Challenges (Using Built-In Providers Only)

🧩 Challenge 1: Create a Random Password (random provider)

Requirement:

- Create a variable for password length
- Use `random_password` to generate password
- Output the password

Hints:

```
variable "pwd_length" {
  type = number
}

resource "random_password" "mypwd" {
  length = var.pwd_length
}

output "generated_password" {
  value     = random_password.mypwd.result
  sensitive = true
}
```

🧩 Challenge 2: Create a local file from user input (local provider)

Requirement:

- Ask user for filename
- Ask user for message
- Create file dynamically using `local_file`

🧩 Challenge 3: Create 3 files using list variable

Requirement:

- Create a list variable containing 3 filenames
- Use `for_each` loop on `local_file` resource
- Each file should contain text "Hello Terraform"

🧩 Challenge 4: Validate Memory Size

Requirement:

Create a variable `memory` that:

- Must be even number
- Between 4 and 64

Use validation block.

🧩 Challenge 5: Use object variable to define user details

Object contains:

```
name = string
age  = number
role = string
```

Create a local file outputting user details.

🧠 12. Interview Questions (Intermediate → Advanced)

Basic Level

1. What is a variable in Terraform and why do we use it?
2. How do you assign values to Terraform variables?

Intermediate Level

3. Explain variable precedence in Terraform.
4. Difference between `variables`, `locals`, and `outputs`.
5. What is a sensitive variable?
6. What is the difference between list and tuple types?

Advanced Level

7. How does Terraform handle type constraints internally?
8. How do you dynamically load different variable files based on Terraform workspace?
9. Explain a scenario where variable validation saved you from production failure.
10. Can Terraform variables be computed dynamically at runtime? Why or why not?
11. If two variable definitions conflict, how does Terraform resolve it?
12. How do you use `terraform.tfvars` and environment variables together?
13. Explain complex variable objects in a real project.
14. How do you secure secrets in Terraform without a secret manager?
15. Explain the limitations of variable inheritance in modules.

🎯 13. Summary

This README covered:

- ✔ Deep variable concepts
- ✔ Types (primitive + complex)
- ✔ Precedence, validation, sensitive handling
- ✔ Advanced use cases
- ✔ Real-world analogies
- ✔ Hands-on labs
- ✔ Interview questions